

Experiment 06 – Advanced SQL Queries

Aim

To implement SQL queries involving conditional aggregation, grouping, subqueries, and hierarchical data classification.

Objective

- To calculate daily differences between two types of sales.
- To use conditional aggregation to process grouped records.
- To understand hierarchical relationships using parent and child nodes.
- To classify nodes as Root, Inner, or Leaf using SQL conditions.
- To sort the generated results according to the given requirements.

Question 1

A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges.

Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day.

The difference should be calculated as:

$$Difference = ApplesSold - OrangesSold$$

Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.

Answer

```
1 SELECT
2     sale_date,
3     SUM(
4         CASE
5             WHEN fruit = 'apples' THEN sold_num
6             WHEN fruit = 'oranges' THEN -sold_num
7         END
8     ) AS diff
9 FROM Sales
10 GROUP BY sale_date
11 ORDER BY sale_date;
```

Output

Schema SQL

```

1 CREATE TABLE Sales
2 (
3   sale_date DATE,
4   fruit VARCHAR(20),
5   sold_num INT,
6
7   PRIMARY KEY (sale_date, fruit)
8 );
9
10 INSERT INTO Sales
11 VALUES
12 ('2020-05-01','apples',10),
13 ('2020-05-01','oranges',8),
14
15 ('2020-05-02','apples',15),
16 ('2020-05-02','oranges',15);

```

Text to DDL

Query SQL

```

1 select sum(case when fruit = 'apples' then sold_num
2           else -sold_num end)
3 from sales
4 group by sale_date
5 order by sale_date

```

Results

Query #1 Execution time: 1.79ms

sum
2
0
20
-1

Copy as Markdown

Question 2

A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where:

- id represents the current node.
- p_id represents the parent node.

Classify every node into one of the following categories:

- **Root:** A node that does not have any parent.
- **Leaf:** A node that has a parent but does not have any children.
- **Inner:** A node that has both a parent and at least one child.

Display the node ID along with its category. Return the result sorted by node ID.

Answer

```

1 SELECT
2   id,
3   CASE
4     WHEN p_id IS NULL THEN 'Root'
5     WHEN id IN (
6       SELECT p_id
7       FROM Tree
8       WHERE p_id IS NOT NULL
9       GROUP BY p_id
10    ) THEN 'Inner'
11    ELSE 'Leaf'
12  END AS Type
13 FROM Tree
14 ORDER BY id;
15
16 ----- or
17 select id,
18 case when
19 p_id is NULL then "Root"
20 when id in (Select p_id from tree) then "Inner"
21 else "Leaf"
22 end as type
23 From tree;
24 --- use sub query

```

Output

608. Tree Node

Medium Topics Companies Hint

SQL Schema > Pandas Schema >

Table: Tree

Column Name	Type
id	int
p_id	int

id is the column with unique values for this table. Each row of this table contains information about the id of a node and the id of its parent node in a tree. The given structure is always a valid tree.

Each node in the tree can be one of three types:

- **"Leaf"**: if the node is a leaf node.
- **"Root"**: if the node is the root of the tree.
- **"Inner"**: If the node is neither a leaf node nor a root node.

Write a solution to report the type of each node in the tree.

Return the result table in **any order**.

The result format is in the following example.

```

1 select id,
2 case when
3 p_id is NULL then "Root"
4 when id in (Select p_id from tree) then "Inner"
5 else "Leaf"
6 end as type
7 from tree;
8 --- use sub query

```

Saved

Testcase > Test Result

Accepted Runtime: 189 ms

Case 1 Case 2

Input

Tree =

id	p_id
1	null
2	1
3	1
4	2
5	2

608. Tree Node

Medium Topics Companies Hint

SQL Schema > Pandas Schema >

Table: Tree

Column Name	Type
id	int
p_id	int

id is the column with unique values for this table. Each row of this table contains information about the id of a node and the id of its parent node in a tree. The given structure is always a valid tree.

Each node in the tree can be one of three types:

- **"Leaf"**: if the node is a leaf node.
- **"Root"**: if the node is the root of the tree.
- **"Inner"**: If the node is neither a leaf node nor a root node.

Write a solution to report the type of each node in the tree.

Return the result table in **any order**.

The result format is in the following example.

```

1 select
2 id,
3 case
4 when p_id is null then 'Root'
5 when id in (
6 select p_id
7 from Tree
8 where p_id is not null
9 group by p_id
10 ) then 'Inner'
11 else 'Leaf'
12 end as type
13 from Tree

```

Saved

Testcase > Test Result

Accepted Runtime: 90 ms

Case 1 Case 2

Input

Tree =

id	p_id
1	null
2	1
3	1
4	2
5	2

Learning Outcome

After completing this experiment, the learner will be able to:

- Use conditional aggregation to calculate values from grouped records.
- Apply CASE expressions to perform conditional classification.
- Use GROUP BY and ORDER BY effectively.
- Identify parent-child relationships in hierarchical data.
- Classify hierarchical nodes as Root, Inner, or Leaf using SQL.
- Use subqueries to determine whether a node acts as a parent.